

Conhecer as características da linguagem JavaScript

A partir do nome da linguagem podemos identificar que a linguagem JavaScript foi originalmente projetada para ser baseada em script, porém, atualmente ela se tornou uma linguagem de programação de propósito geral.

Algumas características da linguagem JavaScript a seguir.

- É uma linguagem de programação projetada para a Web, possibilitando a construção de páginas web interativas, respondendo a ações do usuário e modificando o conteúdo da página de forma dinâmica.
- É uma linguagem interpretada, de forma que o código-fonte é executado diretamente pelo navegador, sem a necessidade de compilação prévia.
- É considerada uma linguagem de alto nível e multiparadigma, suportando estilos como a programação orientada a objetos e a programação funcional.
- É possível incorporar o código JavaScript diretamente em seu HTML ou vinculá-lo a um arquivo externo.
- É uma linguagem tipada dinamicamente, o que significa que você não precisa declarar o tipo de uma variável explicitamente. O tipo de uma variável é determinado em tempo de execução.
- Embora seja executado nos navegadores do lado do cliente, a linguagem também pode ser executada do lado do servidor, por meio de tecnologias como o Node.js e a Apache CouchDB.
- Pode ser usado para acessar e modificar o conteúdo de um documento HTML, permitindo a criação de páginas web dinâmicas mencionadas anteriormente.
- Também pode ser usado para manipular estilos CSS, permitindo a alteração dinâmica da aparência de elementos HTML.
- Possui atualmente uma ampla gama de bibliotecas e frameworks, aumentando ainda mais a popularidade e funcionalidades da linguagem.

1. Linguagem interpretada

JavaScript não precisa ser compilado antes de rodar. Ele é executado diretamente pelo navegador.

- Desenvolvimento mais rápido
- Testes imediatos

2. Tipagem dinâmica

Você não precisa declarar o tipo da variável. Isso dá mais flexibilidade, mas exige cuidado para evitar erros.

```
let x = 10; // número
x = "texto"; // agora virou string
```

3. Baseado em protótipos (orientação a objetos)

Diferente de outras linguagens, ele usa **protótipos** em vez de classes tradicionais (apesar de hoje existirem class também).

```
function Pessoa(nome) {
  this.nome = nome;
}
Pessoa.prototype.falar = function() {
  console.log("Olá!");
};
```

4. Assíncrono e não bloqueante

JavaScript consegue executar várias tarefas sem ficar esperando uma terminar para começar outra.

Quando uma tarefa demorada é encontrada ele “entrega” esta tarefa para o ambiente (navegador ou Node), e continua a execução. Para isso ele utiliza um recurso chamado Event Loop.

O Event Loop fica verificando:

1. “Tem alguma tarefa pronta para executar?”
2. Se sim, executa

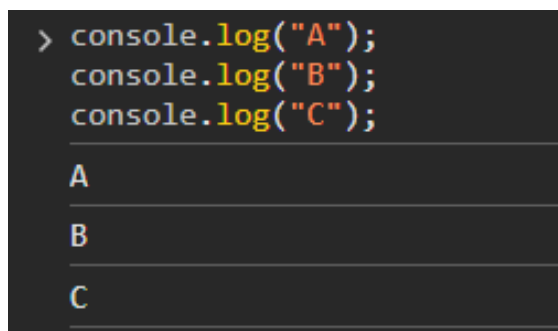
3. Se não, continua esperando.

Por exemplo:

- Requisições de API
- Temporizadores

Código síncrono, que executa linha por linha, esperando cada passo terminar:

```
console.log("A");  
console.log("B");  
console.log("C");
```



```
> console.log("A");  
console.log("B");  
console.log("C");  
A  
B  
C
```

Código assíncrono, que permite "pular" partes demoradas e continuar:

```
console.log("A");  
setTimeout(() => {  
  console.log("B");  
}, 2000);  
console.log("C");
```



```
>  
console.log("A");  
  
setTimeout(() => {  
  console.log("B");  
}, 2000);  
  
console.log("C");  
A  
C  
← undefined  
B
```

Executa código síncrono	Call Stack: console.log("A") → executa
Mostra	A
Encontra setTimeout	setTimeout → vai para Web APIs

Continua execução	<code>console.log("C")</code>
Mostra	C
Após 2 segundos	callback vai para a fila (queue)
Event Loop entra em ação	
Verifica	Stack está vazia?
Então	Move a função para a Stack
Executa all-back	<code>console.log("B")</code>
Mostra	B

[Call Stack]	[Web API]	[Queue]
A	timer	-
↓		
C	timer	-
↓		
(empty)	timer	B
↓		
B		

Importante estudar mais:

- Requisições para API
- Temporizadores
- Leitura de arquivos
- Entender a diferença entre:
 - Callbacks
 - Promises
 - Async/await
- <https://dev.to/altencirsilvajr/uso-assincrono-entenda-a-diferenca-sobre-callbacks-promises-e-asyncawait-439l>
- https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Statements/async_function
- <https://nodejs.org/learn/asynchronous-work/javascript-asynchronous-programming-and-callbacks>

5. Executa no navegador (client-side)

Ele roda diretamente no navegador do usuário, e permite:

- Interações rápidas sem recarregar a página
- Melhor experiência do usuário

6. Também roda no servidor (Node.js)

Com Node.js, JavaScript também pode ser usado no backend., ou seja, uma linguagem só para frontend e backend

7. Multiplataforma

Funciona em:

- Navegadores
- Servidores
- Aplicações mobile (React Native)
- Desktop (Electron)

8. Sintaxe simples e flexível

```
let nome = "Ana";  
console.log(nome);
```

9. Grande ecossistema

Tem milhares de bibliotecas e frameworks:

- React → interfaces
- Angular → apps grandes
- Vue → simplicidade
- Express → servidores

10. Manipulação do HTML e CSS (DOM)

JavaScript pode alterar a página em tempo real.

```
document.getElementById("titulo").innerText = "Novo texto";
```

Confira as diversas funcionalidades que a linguagem oferece:

- Manipulação do Document Object Model (DOM): JavaScript pode ser usada para acessar, modificar e manipular elementos HTML e CSS em tempo real, permitindo criar páginas web dinâmicas, como alterar o conteúdo, estilos, classes e atributos de elementos na página.
- Resposta a Eventos: com JavaScript você pode responder a eventos do usuário, como cliques e movimento do mouse.
- Validação de Formulários: você pode usar JavaScript para validar dados de entrada do usuário em formulários, antes de serem enviados para o servidor.
- Requisições Ajax (Assíncronas): JavaScript permite fazer requisições assíncronas a servidores, permitindo a atualização de partes específicas de uma página sem recarregar a página inteira.
- Armazenamento Local: JavaScript permite usar recursos de armazenamento local, como localStorage para armazenar informações no navegador do usuário.
- Animações e Efeitos: JavaScript permite criar animações e efeitos visuais, como transições e animações de gráficos SVG.
- Integração de APIs Externas: JavaScript possibilita se integrar a APIs externas, como serviços de mapas, redes sociais e feeds de notícias.
- Criação de Jogos Web: JavaScript é usado para criar jogos web, desde jogos simples em 2D até experiências mais complexas e envolventes. Nesse contexto, existem até competições específicas para games criados via HTML, CSS e JavaScript, como o js13kGames – <https://js13kgames.com>
- JavaScript também pode ser utilizado para visualização de dados. Veja um exemplo de uma biblioteca JavaScript para visualização dinâmica de dados, chamada D3.js – <https://observablehq.com/@d3>

Compreender as formas de uso da linguagem JavaScript

Via console do navegador

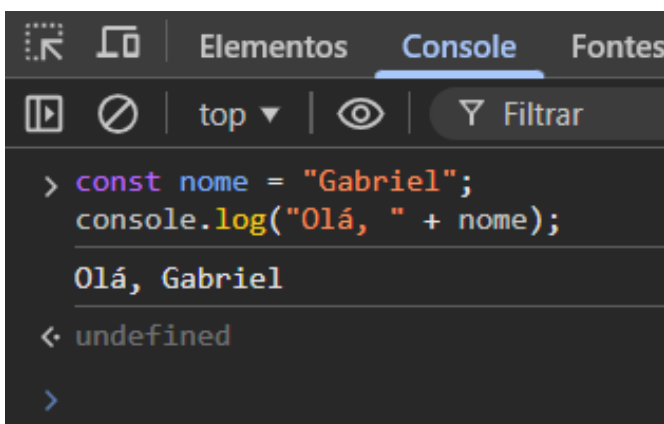
Uma das formas para se utilizar o JavaScript é a partir do console do navegador. Essa ferramenta é utilizada para testar e entender o comportamento do seu código JavaScript. Ela está disponível nas ferramentas de desenvolvimento dos navegadores, podendo ser acessada clicando com o botão direito do mouse na página e selecionando “Inspecionar” e selecionando a aba “Console”. Outra opção é acessar pelas teclas de atalho “Ctrl + Shift + J”.

PRATICANDO:

- Abra uma página web qualquer e clique com o botão direito do mouse na página, selecionando a opção Inspecionar;
- Após isso, acesse a ferramenta Console, conforme a figura a seguir. Ou use as teclas de atalho “Ctrl + Shift + J” para ir diretamente ao console.
- No console para a inserção de código JavaScript. Como exemplo, utilize o console inserindo o método `console.log()` para exibir mensagens no console do navegador, conforme exemplo a seguir:

```
const nome = "Gabriel"; <pressione Shift+ENTER para ir para a próxima linha>
```

```
console.log("Olá, " + nome); <pressione ENTER para "executar" os comandos digitados>
```



`console.log` refere-se a um método estático que apresenta uma mensagem de saída ao console.

Existem outras variações que podem ser utilizadas. Além do `log`, você pode usar as seguintes opções:

- `console.log("mensagem")`

- `console.info("mensagem")`
- `console.debug("mensagem")`
- `console.warn("mensagem")`
- `console.error("mensagem")`

Você pode também utilizar o console para executar comandos JavaScript diretamente no navegador.

Como exemplo, digite o código a seguir que emite um alerta ao usuário e pressione "Enter" para executá-lo:

```
alert("Olá, mundo")
```

Outro exemplo é utilizar o método `prompt()` para capturar uma entrada de dados de um usuário, conforme exemplo a seguir.

```
Var name = prompt("Informe seu nome:");  
alert("Olá " + name);
```

Incorporação direta em arquivo HTML

Uma outra opção possível é utilizar o código JavaScript em seu documento HTML. Isso pode ser feito utilizando a tag `<script>` dentro do arquivo HTML.

Você pode adicionar essas tags no `<head>` ou antes do fechamento do `</body>`, conforme exemplos a seguir.

```
<!DOCTYPE html>  
<html>  
<head>  
  <script type="text/javascript">  
    function saudacao() {  
      alert("Olá, mundo!");  
    }  
  </script>  
</head>  
<body>  
  <button onclick="saudacao()">Clickar</button>  
</body>  
</html>
```



```

<!DOCTYPE html>
<html>
<head>
</head>
<body>
  <button onclick="saudacao()">Clicar</button>
  <script type="text/javascript">
    function saudacao() {
      alert("Olá, mundo!");
    }
  </script>
</body>
</html>

```

Uso de arquivo externo

Uma boa prática também é a utilização de arquivo externo, com extensão .js, para incorporar seu código JavaScript. Você pode vincular esses arquivos externos ao documento HTML usando a tag <script>, conforme exemplo a seguir. Esse script pode estar dentro da tag <head> ou também na tag <body>.

Script.js	index.html
<pre> function audação() { alert("Olá, mundo!"); } </pre>	<pre> <!DOCTYPE html> <html> <head> <script src="script.js"></script> </head> <body> <button onclick="saudacao()">Clicar</button> </body> </html> </pre>

Tipos de variáveis

Para declarar variáveis em JavaScript, podem ser utilizadas as palavras-chaves var, let ou const. No decorrer das aulas de JavaScript veremos mais informações sobre tais possibilidades, mas, em resumo: var possui escopo de função, let e const possuem escopo de bloco. Além disso, var e let permitem reatribuição, enquanto const declara variáveis imutáveis após a inicialização (constante).

Além disso, conforme mencionado, JavaScript é uma linguagem de tipagem dinâmica, ou seja, você não precisa declarar o tipo da variável explicitamente. Os tipos de variáveis em JavaScript incluem

- Número (number): representa inteiros e números de ponto flutuante.
 - `var idade = 25;`
 - `const preco = 14.44;`
 - `let valor = 50;`
- String: para armazenar texto.
 - `let nome = "Joana";`
 - `let frase = 'Aprendendo JavaScript';`
- Boolean: verdadeiro (true) ou falso (false).
 - `var fraude = true;`
 - `let desligado = false;`
- Array: lista de valores, acessados por índices.
 - `let frutas = ["maçã", "uva", "laranja"];`
 - `let itens = [1, 2, 3, 4, 5]`
- Objeto: para armazenar propriedades e métodos.
 - `let pessoa = {
 nome: "Joana",
 idade: 44,
 cidade: "Londrina"
};`
- Função: para ser invocada para executar ações.
 - `function cumprimento(nome) {
 return `Olá, ${nome}!`;
}`
- null: ausência de qualquer valor ou objeto. Devido ao fato de JavaScript ser case-sensitive, null não é o mesmo que Null e NULL.
 - `let valorNulo = null;`
- Undefined: variável declarada, mas não inicializada.
 - `let valor;`

Comentários

Existem duas formas principais de adicionar comentários em JavaScript: comentários de uma linha e comentários de várias linhas, conforme exemplo a seguir.

```
// Comentário de uma linha.  
let idade = 42; // Comentário na mesma linha de código  
/*  
comentário  
de várias  
linhas.  
*/
```

Operadores aritméticos

JavaScript oferece diversos operadores aritméticos para realizar operações matemáticas em números.

- Adição (+) – *também é utilizado como operador de concatenação de strings*
- Subtração (-)
- Multiplicação (*)
- Divisão (/)
- Resto da divisão (%)
- Incremento (++)
- Decremento (--)

Os operadores:

- Subtração (-)
- Multiplicação (*)
- Divisão (/)

Nunca fazem concatenação. Eles sempre tentam converter tudo para número.

```
console.log("8" * "2");    // 16  
console.log("8" - "3");    // 5  
console.log("8" / "2");    // 4  
console.log("8" + "2");    // "82"  
console.log("texto" / 2);  // NaN → Not a Number  
console.log("oi" * 3);     // NaN → Not a Number
```

```
> console.log("8" * "2");  
console.log("8" - "3");  
console.log("8" / "2");  
console.log("8" + "2");  
console.log("texto" / 2);  
console.log("oi" * 3);
```

16

5

4

82

NaN

NaN

O operador de adição (+) pode realizar duas operações diferentes:

- Soma (adição)

```
let a = 5;  
let b = 3;  
console.log(a + b); // 8
```

- Concatenação (junção de textos)

```
let nome = "João";  
let idade = 20;  
console.log(nome + idade);
```

Importante:

1. Se tiver string → tudo vira string

```
console.log("5" + 3); // "53"
```

2. JavaScript converte automaticamente (coerção de tipo)

```
console.log("10" + 5); // "105"
```

3. Ordem importa!

```
console.log(2 + 3 + "4");
```

2 + 3 = 5

5 + "4" = "54"

```
console.log("2" + 3 + 4);
```

"2" + 3 = "23"

"23" + 4 = "234"

4. Como forçar adição

```
console.log(Number("5") + 10);
```

ou

```
console.log(Number("5") + 10);
```

5. Como forçar concatenação

```
console.log(String(10) + 5);
```

Exemplos:

```
console.log(5 + 5);      // 10
console.log("5" + 5);    // "55"
console.log(5 + "5");    // "55"
console.log("5" + "5");  // "55"
```

Montar mensagens:

Uso comum:

```
let nome = "Ana";
console.log("Olá, " + nome + "!");
```

Template string:

```
let nome = "Ana";
console.log(`Olá, ${nome}!`);
```

O operador de divisão (/) sempre realiza uma divisão inteira. Em JavaScript um número é sempre tratado como “Number” (ponto flutuante). Para obtermos o quociente da divisão inteira utilizamos:

Math.floor() - sempre arredonda para baixo:

```
let a = 10;
let b = 3;
let resultado = Math.floor(a / b);
console.log(resultado); // 3

console.log(Math.floor(7 / 2)); // 3
console.log(Math.floor(-7 / 2)); // -4
```

Math.trunc() - remove a parte decimal (não arredonda)

```
let resultado = Math.trunc(10 / 3);
console.log(resultado); // 3
```

Importante:

```
console.log(Math.floor(-7 / 2)); // -4  
console.log(Math.trunc(-7 / 2)); // -3
```

Operadores de comparação

Os operadores de comparação são usados para comparar valores e expressões, geralmente resultando em um valor booleano, verdadeiro (true) ou falso (false). Veja exemplos a seguir:

- Igual (==): verifica se dois valores são iguais.
 - `5 == 5; // true`
 - `"10" == 10; // true`
- Estritamente Igual (===): verifica se dois valores são iguais e do mesmo tipo de dados.
 - `5 === 5; // true`
 - `"10" === 10; // false`
- Diferente (!=): verifica se dois valores são diferentes.
 - `5 != 3; // true`
 - `"5" != 5; // false` (conversão de tipo implícita)
- Estritamente Diferente (!==): verifica se dois valores são diferentes ou de tipos diferentes.
 - `5 !== "5"; // true`
 - `5 !== 5; // false`
- Maior que (>): verifica se o valor da esquerda é maior que o valor da direita.
 - `10 > 5; // true`
- Menor que (<): verifica se o valor da esquerda é menor que o valor da direita.
 - `5 < 10; // true`
- Maior ou Igual a (>=): verifica se o valor da esquerda é maior ou igual ao valor da direita.
 - `10 >= 5; // true`
- Menor ou Igual a (<=): verifica se o valor da esquerda é menor ou igual ao valor da direita.
 - `5 <= 10; // true`
- Operador Ternário (?:): operador de comparação que permite criar expressões condicionais compactas.

- Sintaxe:

condição ? valor_se_verdadeiro : valor_se_falso

```
> let idade = 18;
let mensagem = idade >= 18 ? "Maior de idade" : "Menor de idade";
console.log(mensagem);

Maior de idade
```

```
> let numero = 10;
let tipo = numero % 2 === 0 ? "Par" : "Ímpar";
console.log(tipo);
console.log(5 > 3 ? "Sim" : "Não");

Par
Sim
```

Operadores lógicos

Tais operadores são usados para combinar expressões booleanas anteriormente apresentadas.

Existem três operadores lógicos principais: && (E lógico), || (OU lógico) e ! (NÃO lógico). Veja alguns exemplos:

Operador && - Retorna true se ambas as expressões forem verdadeiras.

```
const idade = 25;
const possuiCarteiraDeMotorista = true;
if (idade >= 18 && possuiCarteiraDeMotorista) {
  console.log("Pode dirigir.");
}
else {
  console.log("Não pode dirigir.");
}
```

Operador || - Retorna true se pelo menos uma das expressões for verdadeira.

```
const temCartaoDeCredito = true;
const temDinheiro = false;
if (temCartaoDeCredito || temDinheiro) {
  console.log("Pode fazer a compra.");
} else {
  console.log("Não pode fazer a compra.");
}
```

Operador ! - Inverte o valor de uma expressão booleana

```
const usuarioLogado = true;
if (!usuarioLogado) {
  console.log("Faça o login para continuar.");
} else {
  console.log("Bem-vindo!");
}
```

Estruturas de controle

Vimos nos exemplos anteriores o uso do if, como estrutura de controle condicional. Além dessas opções, também podem ser utilizadas os seguintes operadores de controle:

else-if - Permite verificar múltiplas condições em sequência.

```
const diaDaSemana = "terça";
if (diaDaSemana === "sábado" || diaDaSemana === "domingo") {
  console.log("É fim de semana!");
} else if (diaDaSemana === "segunda" || diaDaSemana === "terça") {
  console.log("É início da semana.");
} else {
  console.log("É um dia comum.");
}
```

while - Usado para criar loops enquanto uma condição for verdadeira.

```
let contador = 0;
while (contador < 5) {
  console.log("Contagem: " + contador);
  contador++;
}
```

do...while - Garante que o bloco de código seja executado pelo menos uma vez, mesmo se a condição for falsa.

```
let numero = 1;
do {
  console.log("Número: " + numero);
  numero++;
} while (numero <= 3);
```


for - Usado para criar loops com um contador.

```
for (let i = 0; i < 5; i++) {  
  console.log("Iteração " + i);  
}
```

for...in - Alternativa para criar loop. Executa iterações a partir de uma variável informada, permitindo percorrer as propriedades de um objeto.

```
var produtos =  
  [  
    { id: 1, tipo: 'camiseta', quantidade: 20 },  
    { id: 2, tipo: 'livro', quantidade: 15 }  
  ]  
var qtdtotal = 0  
for (const i in produtos) {  
  qtdtotal += produtos[i].quantidade  
}
```

switch - Usado para fazer múltiplas verificações em uma única variável.

```
let fruta = "maçã";  
switch (fruta) {  
  case "maçã":  
    console.log("É uma maçã.");  
    break;  
  case "banana":  
    console.log("É uma uva.");  
    break;  
  default:  
    console.log("Não está na lista.");  
}
```

PRATICANDO:

- Crie um arquivo index.html e insira o código a seguir, visualizando o resultado a partir do Live Server.
- Nesse exemplo, foi utilizado document.write(), um método em JavaScript que permite escrever conteúdo diretamente no documento HTML a partir do código JS. Posteriormente veremos outras alternativas mais eficientes para essa ação.

```
<!DOCTYPE html>
<html lang="pt-br">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Aprendendo Javascript</title>
</head>
<body>
  <script type="text/javascript">
    var i = 0;
    for (i = 0; i <= 8; i++) {
      document.write("Número digitado: " + i);
      document.write("<br />");
    }
  </script>
</body>
</html>
```

Objeto Date

O objeto Date é usado em JavaScript para trabalhar com datas e horários. Veja alguns dos recursos a partir do exemplo a seguir:

```
const dataAtual = new Date();
const dia = dataAtual.getDate(); // Dia do mês (1-31)
const mes = dataAtual.getMonth(); // Mês (0-11, janeiro é 0)
const ano = dataAtual.getFullYear();
const hora = dataAtual.getHours(); // Hora (0-23)
const minutos = dataAtual.getMinutes(); // Minutos (0-59)
const segundos = dataAtual.getSeconds();
alert(`Data atual: ${dia}/${mes + 1}/${ano}`);
console.log(`Horário atual: ${hora}:${minutos}:${segundos}`);
```

```
const dataAtual = new Date();
const dia = dataAtual.getDate(); // Dia do mês (1-31)
const mes = dataAtual.getMonth(); // Mês (0-11, janeiro é 0)
const ano = dataAtual.getFullYear();
const hora = dataAtual.getHours(); // Hora (0-23)
const minutos = dataAtual.getMinutes(); // Minutos (0-59)
const segundos = dataAtual.getSeconds();
alert(`Data atual: ${dia}/${mes + 1}/${ano}`);
console.log(`Horário atual: ${hora}:${minutos}:${segundos}`);
```

Nesse exemplo, o conteúdo dentro de `alert()` foi implementado usando o conceito chamado "Template strings", no qual o conteúdo é envolvido por `` `` em vez de aspas simples ou duplas. Esse recurso permite utilizar placeholders que são indicados por um cifrão seguido de chaves (exemplo: `${dia}`)

Veja as outras funções do Objeto Date por meio desse link do MDN -

https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Global_Objects/Date

Template Strings

As Template Strings (ou Template Literals) são uma forma moderna de trabalhar com strings em JavaScript, introduzida no ES6 (2015). Elas facilitam a criação de textos dinâmicos e organizados, substituindo a antiga necessidade de concatenar partes com o operador +.

Principais Características

- **Delimitadores:** São escritas entre crases (`) em vez de aspas simples ou duplas.
- **Interpolação:** Permite inserir variáveis ou expressões diretamente no texto usando a sintaxe `${expressão}`.
- **Múltiplas Linhas:** Você pode quebrar linhas naturalmente dentro do código sem precisar de caracteres de escape como `\n`

Interpolação de Variáveis

Em vez de juntar textos e variáveis com +, você faz assim:

```
const nome = "Alice";
const idade = 25;
// Exemplo com Template String
console.log(`Olá, meu nome é ${nome} e tenho ${idade} anos.`);

// Saída: Olá, meu nome é Alice e tenho 25 anos.
```

Expressões Matemáticas e Lógicas

Você pode realizar cálculos ou operações dentro das chaves `${}`:

```
const preco = 10;
const taxa = 0.5;
console.log(`O total é: ${preco + taxa}`);

// Saída: O total é: 10.5
```

Strings de Múltiplas Linhas

Muito útil para gerar blocos de código HTML ou mensagens longas:

```
const conteudo = `  
  <div>  
    <h1>Título</h1>  
    <p>Este é um parágrafo em múltiplas linhas.</p>  
  </div>  
`;  
;
```

Onde você pode ver mais sobre JavaScript:

- https://developer.mozilla.org/pt-BR/docs/Learn_web_development/Core/Scripting
- <https://www.alura.com.br/artigos/javascript#o-que-e-javascript-para-que-serve-a-linguagem-javascript-e-onde-e-usada>
- <https://learn.microsoft.com/pt-br/shows/beginners-series-to-javascript/>
- <https://www.freecodecamp.org/>
- <https://me.developers.google.com/learning?language=e:language:javascript>